

Day-19 | Quantum Cryptography

Secure Communication & BB84 Protocol

1. Learning Objectives

- ✓ Understand the principles of Quantum Cryptography vs. Classical Cryptography.
- ✓ Explain the intuition behind the **BB84 Quantum Key Distribution (QKD)** protocol.
- ✓ Understand how eavesdropping is physically detectable using quantum measurements.
- ✓ Implement a simplified QKD simulation using Qiskit.

2. Quick Recap

- **Measurement:** Measuring a quantum state forces it to collapse; the original state is lost.
- **No-Cloning Theorem:** It is physically impossible to perfectly copy an unknown quantum state.

Transition: Today, we use these physical constraints not for computation, but for **security**. We will use qubits to generate unbreakable secret keys.

3. Why Do We Need Quantum Cryptography?

Motivation: Classical encryption (like RSA) relies on complex math (e.g., factoring large primes). A powerful quantum computer (running Shor's algorithm) will eventually break these locks. Quantum Cryptography provides a future-proof solution: security guaranteed by the **laws of physics**, not mathematical difficulty.

Real-World Applications

- **Secure Banking:** Transferring encryption keys for high-value financial data.
- **Government Comms:** Protecting national security data from future quantum decryption.
- **Data Center Links:** Securing traffic between distributed cloud servers.

4. Core Concept: QKD does NOT send the message

What is QKD?

Quantum Key Distribution (QKD) does *not* encrypt the actual message (like an email). It securely distributes a **random sequence of bits (a Key)**. Once Alice and Bob share this completely secure key, they use standard classical encryption to lock the actual message.

5. Concept Expansion: Classical vs. Quantum Security

Classical Cryptography

Security relies on computational limits (math).

Eve can secretly copy the data and decrypt it later.

Interception is undetectable.

Quantum Cryptography (QKD)

Security relies on the laws of physics.

No-Cloning: Eve cannot copy the qubit.

Measurement Disturbs: If Eve looks at the qubit, the state collapses, alerting Alice and Bob.

6. Core Mechanism: The BB84 Protocol Workflow

The BB84 Protocol involves Alice (Sender), Bob (Receiver), and Eve (Eavesdropper).

1. **Alice Encodes:** Alice generates random bits (0 or 1). She randomly encodes them using one of two bases: **Computational (+)** or **Diagonal ($\times$)**.
2. **Transmission:** She sends the photons (qubits) to Bob over a quantum channel.
3. **Bob Measures:** Bob doesn't know Alice's bases. He randomly guesses a basis (+ or $\times$) to measure each photon.
4. **Classical Chat:** Over a normal, public channel, Alice and Bob share *only the bases* they used (not the bits).
5. **The Key:** They keep only the bits where their bases **matched**. This is the secret key.

7. Visualizing BB84 Basis Matching

Step	Bit 1	Bit 2	Bit 3	Bit 4	Bit 5	Bit 6
Alice's Bit	1	0	1	1	0	0
Alice's Basis	+	×	+	×	+	×
Bob's Basis	+	+	×	×	×	×
Match?	Yes	No	No	Yes	No	Yes
Shared Key	1	-	-	1	-	0

8. Eavesdropping Detection (QBER)

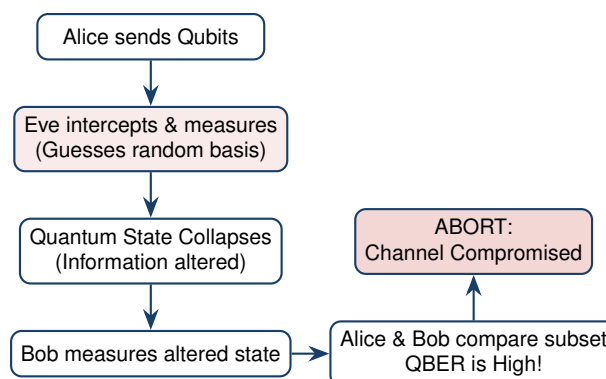
How do they know if Eve listened? They publicly compare a small chunk of their shared key.

Quantum Bit Error Rate (QBER):

$$\text{QBER} = \frac{\text{Number of Incorrect Bits}}{\text{Total Compared Bits}}$$

If Eve measures the photons, she uses the wrong basis 50% of the time, collapsing the state incorrectly. When Bob receives it, he gets the wrong bit. If QBER > threshold (e.g., 11%), they abort and start over.

9. Detecting Eve (Flowchart)



10. Practical Implementation: Qiskit BB84 Simulation

This simple Python snippet simulates the core logic of basis matching.

```

from qiskit import QuantumCircuit
from qiskit.primitives import StatevectorSampler
import random

# 1. Random Generation (Simulating 8 bits)
alice_bits = [random.randint(0,1) for _ in range(8)]
alice_bases = [random.randint(0,1) for _ in range(8)] # 0: +, 1: x
bob_bases = [random.randint(0,1) for _ in range(8)] # 0: +, 1: x

```

```
circuits = []
# 2. Alice Encodes and Bob Measures
for bit, a_basis, b_basis in zip(alice_bits, alice_bases, bob_bases):
    qc = QuantumCircuit(1, 1)

    if bit == 1: qc.x(0)          # Encode Bit
    if a_basis == 1: qc.h(0)      # Apply Diagonal Basis (x)

    # --- Qubits travel to Bob ---

    if b_basis == 1: qc.h(0)      # Bob applies his basis
    qc.measure(0, 0)
    circuits.append(qc)

# 3. Simulate Measurements
sampler = StatevectorSampler()
job = sampler.run(circuits, shots=1)

print("Alice_Bases:", alice_bases)
print("Bob_Bases_": bob_bases)
# In a full protocol, they now compare bases and keep the matching bits.
```

11. ★ Key Takeaways

- > **Quantum Cryptography** uses physical laws to secure communication, immune to future computing power.
- > **QKD (Quantum Key Distribution)** securely shares a random key, it does not transmit the message itself.
- > The **BB84 Protocol** uses random basis selection (+ or ×) to encode bits into photon polarization.
- > The **No-Cloning Theorem** and **Measurement Collapse** guarantee that Eve cannot secretly intercept the key.
- > **QBER (Quantum Bit Error Rate)** acts as an alarm; high errors mean the channel is compromised.